

Full-Stack Battleship

Team S.S. GPT

Anthony Frialde and Christian Johnston

Project Overview:

A persistent, distributed multiplayer server that manages turn-based game logic and player statistics through a secure REST API, along with a high-performance, intuitive web client that allows real-time interaction, multi-game management, and player persistence.

Phase Objective / Problem Definition:

Currently, web-based Battleship implementations often lack persistence and true multiplayer synchronization. Most casual versions are limited to local play or lose state upon browser refresh. The objective of Phase 1 was to build a persistent, distributed multiplayer server that serves as the “source of truth” for all game logic. This phase was completed when the server passed the test suite, handled any number of players correctly in a relational database, and survived a full system restart without losing game state. The objective of Phase 2 now is to transform our API into a polished, playable product.

Target Users:

- Primary User - Player
- User Skill Level - Non-Technical
- Environment - Desktop/Laptop, Mobile

Core Features

1. The game should support any number of players.
2. The game should function as a standard game of Battleship does
3. Game statistics should be tracked.
4. After refresh, the game should keep all of the data.
5. The game should have dynamic lobbies that can be browsed, joined, and created with configurable grid sizes
6. The game should have clear visual indicators for whose turn it is, who has been eliminated, and real-time hits/misses.
7. The client must work with the other servers made by other students.

Non-Goals:

What will not be built:

- Autonomous AI Player
 - No AI Battleship player.

User Stories:

- As a player, I want to be able to keep the game progress so that if my page is refreshed or the browser gets closed accidentally, I can continue my progress.
- As a player, I want to be able to join another player's game so that I can play against them online.
- As a player, I want to be able to see my current stats so that I can keep track of my progress and statistics.
- As a player, I want to see all available games so I can quickly jump into a match.
- As a player, I want a clear visual indicator of my accuracy and Win/Loss ratio to track my improvement.
- As a player, I want to see all opponents' boards in order to strategize my next shot.

Feature Scope / MVP Definition:

Minimum Required Features:

- Multiplayer Support
 - Capability for 1 - N players to join a game.
- Relational Schema
 - Database setup, including a GamePlayers join table
- Turn Rotation
 - Server-side logic to manage player order and eliminate players.
- Dynamic Grid
 - The client can display any grid size returned by the server
- Move Logging
 - Every action must be recorded with a persistent timestamp.
- Player Statistics
 - Track wins, losses, and moves via the API
- Database Persistence
 - Data stays after page refresh and in incognito mode.
- Dynamic Servers
 - The client must work as a browser for all Battleship servers, rather than just our server, so we must call endpoints from a non-hardcoded URL. The user will be able to choose a server to connect to.

Phase 2 of this product will be considered successful when all of these minimum required features are met.

Constraints:

- The backend will run on Render and be written in PHP. The database will be in PostgreSQL, also hosted on Render.
- Game statistics and records will be persistent upon refresh.
- Players cannot choose their game IDs, they will only be assigned by the server.
- Players will not be able to see any game data they shouldn't, such as other players' boards.

Architecture Decisions:

- Database
 - Initialize a relational database (PostgreSQL) with tables for
 - Games
 - Players
 - Moves
 - The GamePlayers join table
- Endpoint Additions
 - Implementation of RESTful endpoints for game creation, joining, move submission, and statistics retrieval
- State Management Approach
 - The client will use interval polling to check the game state while it is not the user's turn. This approach has a simpler, more efficient implementation.
- Multiplayer Considerations
 - Use unique game IDs and the join table to associate multiple players with a single session.
- Turn Logic Considerations
 - The server will validate the "active" player ID against the move history to ensure only the correct player can act.
- Future Proofing
 - All Phase 1 endpoints are designed to remain unchanged for Phase 2 to ensure backward compatibility

API Plan:

GET	• Retrieve Player Statistics • Retrieve Game State • Retrieve Move History
POST	• Reset State • Create Player • Create New Game • Join Existing Game • Place Ships • Fire Shot at Opponent • Restart Game (test mode) • Deterministic Ship Placement (test mode) • Reveal Board State (test mode)
PUT	N/A
DELETE	N/A

Database Plan:

We will utilize a relational model to ensure ACID compliance and handle concurrency.

- Games Table
 - game_id (PK), status (waiting/active/finished), created_at.
- Players Table
 - player_id (PK), username, total_wins, total_losses.
- Moves Table
 - move_id (PK), game_id (FK), player_id (FK), coordinate_x, coordinate_y, result (hit/miss), timestamp.
- GamePlayers (Join Table):
 - Associates players with games; tracks turn_order and is_eliminated.
- Ships Table
 - Stores the coordinates of the ships placed by players

Testing Plan:

- Unit Testing Approach
 - Individual logic functions (Ex: turn rotation, win conditions) will be tested in isolation using a testing framework. We will use the test pool from all the students' submitted tests.
- Regression Testing Approach
 - Once an endpoint is "locked", it will be re-tested after every subsequent code change to ensure no drift.
- Edge-Case Testing Approach
 - Testing for 1-player games, N-player limits, invalid move attempts, and simultaneous join requests
- AI for Testing
 - AI will be prompted to generate test suites based on the API Contract, which we will then manually verify for logic errors

Known Risks / Risk Assessment:

- Schema Migration Risks
 - Changes to the database mid-phase could break existing logic; migrations will be planned in the PRD before implementation
- Turn Logic Risks
 - Errors in turn calculation could lead to "deadlocked" games, where no player is active
- Concurrency Risks
 - Multiple players joining or moving at the same time could cause race conditions; database transactions will be used to ensure ACID compliance.
- Regression Risks
 - Adding stats tracking later in the phase might break the core move-logging logic; re-running Phase 1 tests is mandatory.

AI Usage Plan:

- How AI will be used
 - AI will act as a “Builder”, generating boilerplate code, SQL schema syntax, and unit testing templates
- Decisions Humans Retain
 - Humans own the high-level architecture, the relational model design, and the final approval of all merged code
- Validation of AI Output
 - Every AI-generated snippet must be peer-reviewed by the team and pass the defined test suite before being committed.
- Prompt Log Maintenance
 - A prompt log will be updated for every major feature, documenting the context, the AI's response, and the human critique/modifications